

Estructura de Datos - Módulo 2

Memoria dinámica y estructuras enlazadas orientadas a C++

Aplicación al incidente crítico PAD-Bienestar

Idea clave: Cuando el número de usuarios, sesiones o turnos cambia durante la ejecución, las estructuras dinámicas permiten crear y liberar memoria conforme se necesita. En C++ esto se trabaja principalmente con apuntadores, new, delete, nodos y estructuras enlazadas.

1. Enfoque general del Módulo 2 orientado a C++

El Módulo 2 continúa el camino iniciado con las estructuras lineales estáticas. Ahora el problema principal ya no es solo guardar datos, sino permitir que el programa se adapte cuando la cantidad de información cambia. En el incidente crítico PAD-Bienestar, esto ocurre cuando llegan más personas al punto de acceso digital, se abren nuevas sesiones, se forman turnos o se agregan registros de bitácora sin saber previamente cuántos serán.

Desde el enfoque de Deitel & Deitel en C++ Cómo programar, 9.ª edición, el manejo de arreglos, apuntadores, estructuras, funciones y archivos debe acompañarse de una correcta comprensión de la memoria. En C++, la memoria dinámica se solicita durante la ejecución mediante new y se libera con delete. Esto permite construir estructuras enlazadas capaces de crecer o disminuir según las necesidades del sistema.

Relación con el PDF: El contenido nuclear del Módulo 2 aborda memoria dinámica, stack y heap, apuntadores, listas enlazadas simples y dobles, pilas/colas dinámicas y modularización. En esta versión se adapta la explicación y los ejemplos al lenguaje C++.

Necesidad del PAD-Bienestar	Tema de Módulo 2 en C++	Razón principal
Crear sesiones sin límite fijo	Memoria dinámica	Se reserva memoria cuando llega un usuario.
Ubicar datos en memoria	Apuntadores	Permiten enlazar nodos y acceder a datos.
Guardar usuarios conectados	Lista enlazada simple	Crece y decrece sin arreglo fijo.
Navegar adelante y atrás	Lista doblemente enlazada	Permite recorrido bidireccional.
Consultar última acción	Pila dinámica	Trabaja con lógica LIFO.
Administrar turnos	Cola dinámica	Respeto el orden FIFO.
Organizar el proyecto	Modularización y TDA	Separa declaraciones, implementación y uso.

2. Memoria dinámica en C++

Teoría orientada a C++

La memoria dinámica permite reservar espacio durante la ejecución del programa. A diferencia de un arreglo estático, que necesita definir su tamaño desde el inicio, una estructura dinámica puede crear elementos

conforme se requieren. En C++, esta reserva se realiza con `new` y la liberación con `delete`. El `stack` se usa para variables locales y llamadas a funciones; el `heap` se usa para memoria solicitada manualmente por el programa.

Aplicación PAD-Bienestar: En PAD-Bienestar, esto es útil porque no siempre se sabe cuántas personas llegarán a solicitar servicio. Si el sistema usa memoria dinámica, puede crear una nueva sesión o un nuevo turno cuando la persona llega, y liberar ese espacio cuando la atención termina.

Ejemplo base en C++

```
struct Sesion {  
    int idUsuario;  
    int equipoAsignado;  
    bool activa;  
};  
  
Sesion* nuevaSesion = new Sesion;  
nuevaSesion->idUsuario = 101;  
nuevaSesion->equipoAsignado = 3;  
nuevaSesion->activa = true;  
  
// Cuando la sesión termina:  
delete nuevaSesion;  
nuevaSesion = nullptr;
```

Imagen de apoyo visual

c++ Memoria dinámica en C++

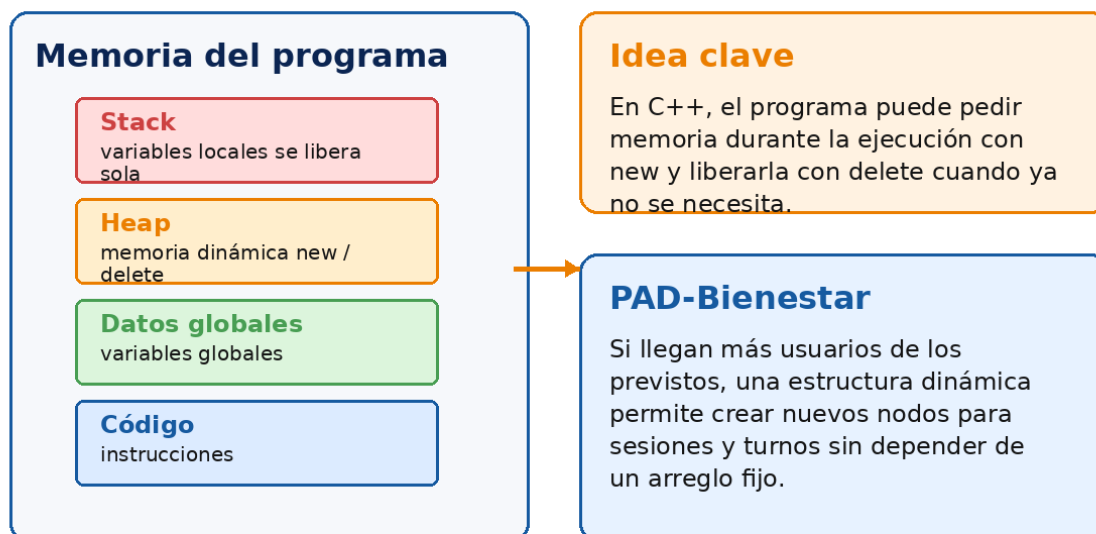


Imagen 1. Memoria dinámica en C++: diferencia visual entre `stack` y `heap` aplicada al crecimiento de sesiones y turnos del PAD-Bienestar.

3. Apuntadores en C++

Teoría orientada a C++

Un apuntador es una variable que guarda una dirección de memoria. En lugar de almacenar directamente el dato, almacena dónde se encuentra ese dato. El operador & obtiene la dirección de una variable; el operador * permite acceder al valor ubicado en esa dirección. En estructuras, el operador -> permite acceder a los campos a través de un apuntador.

Aplicación PAD-Bienestar: En PAD-Bienestar, los apuntadores son necesarios para enlazar nodos de sesiones, turnos o bitácoras. También permiten modificar datos originales desde funciones, por ejemplo, cambiar el estado de un equipo de disponible a ocupado.

Ejemplo base en C++

```
void marcarOcupado(int* estadoEquipo) {
    *estadoEquipo = 1; // cambia el valor original
}

int equipo1 = 0;          // 0 = disponible
marcarOcupado(&equipo1); // ahora equipo1 vale 1
```

Imagen de apoyo visual

c++ Apuntadores en C++

Variable normal

```
int equipo = 3;
valor: 3
```

Apuntador

```
int* p = &equipo;
guarda dirección
```

Operadores

& obtiene la dirección
* accede al valor

Ejemplo PAD-Bienestar: modificar el estado de un equipo

```
void marcarOcupado(int* estadoEquipo) {
    *estadoEquipo = 1; // cambia el valor original
}

int equipo1 = 0;          // disponible
marcarOcupado(&equipo1); // ahora queda ocupado
```

Imagen 2. Apuntadores en C++: uso de &, * y modificación directa de un dato del sistema.

4. Nodos y listas enlazadas simples

Teoría orientada a C++

Una lista enlazada simple es una estructura dinámica formada por nodos. Cada nodo contiene información y un apuntador al siguiente nodo. A diferencia de los arreglos, sus elementos no necesitan estar en posiciones consecutivas de memoria y la lista puede crecer conforme se insertan nuevos nodos.

Aplicación PAD-Bienestar: En PAD-Bienestar, una lista enlazada simple puede guardar las sesiones activas. Cada nodo representa una sesión, con el identificador del usuario, el equipo asignado y el enlace al siguiente registro.

Ejemplo base en C++

```
struct NodoSesion {
    int idUsuario;
    int equipoAsignado;
    NodoSesion* siguiente;
};

NodoSesion* crearNodo(int id, int equipo) {
    NodoSesion* nuevo = new NodoSesion;
    nuevo->idUsuario = id;
    nuevo->equipoAsignado = equipo;
    nuevo->siguiente = nullptr;
    return nuevo;
}
```

Imagen de apoyo visual

c++ Listas enlazadas simples

Lista enlazada simple



Nodo en C++

```
struct NodoSesion {
    int idUsuario;
    int equipoAsignado;
    NodoSesion* siguiente;
};
```

Cada nodo guarda datos de una sesión y una liga hacia el siguiente nodo. La lista crece conforme llegan usuarios; no necesita reservar todos los espacios desde el inicio.

Imagen 3. Lista enlazada simple: cada nodo guarda datos y apunta al siguiente.

5. Inserción, eliminación y recorrido en listas simples

Teoría orientada a C++

Las operaciones básicas de una lista enlazada son insertar, eliminar y recorrer. Insertar implica crear un nuevo nodo y ajustar enlaces. Eliminar requiere desconectar un nodo y liberar su memoria con delete. Recorrer significa avanzar nodo por nodo hasta llegar a nullptr.

Aplicación PAD-Bienestar: En PAD-Bienestar, estas operaciones permiten registrar una nueva sesión, cerrar una sesión activa o mostrar todas las sesiones actualmente registradas.

Ejemplo base en C++

```
void insertarInicio(NodoSesion*& head, int id, int equipo) {
    NodoSesion* nuevo = crearNodo(id, equipo);
    nuevo->siguiente = head;
    head = nuevo;
}

void mostrarSesiones(NodoSesion* head) {
    NodoSesion* actual = head;
    while (actual != nullptr) {
        cout << "Usuario: " << actual->idUsuario
              << " Equipo: " << actual->equipoAsignado << endl;
        actual = actual->siguiente;
    }
}
```

c++ Inserción, eliminación y recorrido

Operaciones básicas

Inserción al inicio

1. Crear nodo nuevo 2. nuevo->siguiente = head 3. head = nuevo

Eliminación

1. Guardar nodo temporal 2. Reacomodar enlaces 3. Liberar con delete

Recorrido

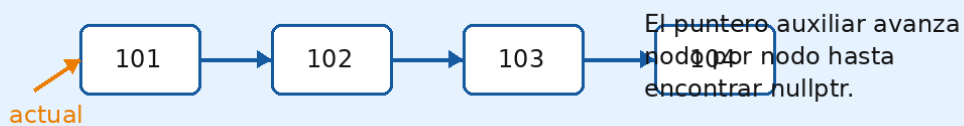


Imagen 4. Operaciones básicas en una lista enlazada simple: insertar, eliminar y recorrer.

6. Listas doblemente enlazadas

Teoría orientada a C++

Una lista doblemente enlazada tiene nodos con dos apuntadores: uno al nodo siguiente y otro al nodo anterior. Esto permite recorrer la estructura hacia adelante y hacia atrás. Su ventaja es la navegación bidireccional; su desventaja es que cada nodo consume más memoria porque guarda un apuntador adicional.

Aplicación PAD-Bienestar: En PAD-Bienestar, podría usarse para consultar un historial de sesiones o eventos en orden normal e inverso, por ejemplo, revisar desde el primer registro del día o desde el evento más reciente.

Ejemplo base en C++

```
struct NodoDoble {
    int idUsuario;
    NodoDoble* anterior;
    NodoDoble* siguiente;
};

void insertarFinal(NodoDoble*& head, NodoDoble*& tail, int id) {
    NodoDoble* nuevo = new NodoDoble{id, tail, nullptr};
    if (tail != nullptr) {
        tail->siguiente = nuevo;
    } else {
        head = nuevo;
    }
}
```

```

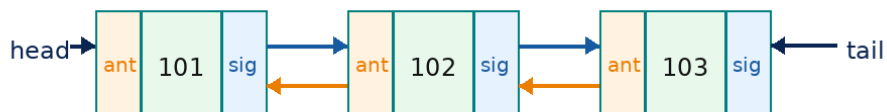
    }
    tail = nuevo;
}

```

Imagen de apoyo visual

c++ Listas doblemente enlazadas

Lista doblemente enlazada



Ventaja visual

Cada nodo conoce al siguiente y al anterior. Por eso se puede avanzar y retroceder, útil para historial, navegación, deshacer/rehacer o consultar sesiones en orden inverso.

Imagen 5. Lista doblemente enlazada: cada nodo apunta al anterior y al siguiente.

7. Pila dinámica

Teoría orientada a C++

Una pila dinámica usa nodos enlazados y sigue el principio LIFO: Last In, First Out. El último elemento insertado es el primero en salir. En una implementación dinámica solo se necesita un apuntador al tope. Insertar y eliminar en el tope son operaciones rápidas.

Aplicación PAD-Bienestar: En PAD-Bienestar, una pila puede registrar acciones recientes del sistema, como el último inicio de sesión, la última asignación de equipo o el último cambio de estado. Es útil para consultar o deshacer la acción más reciente.

Ejemplo base en C++

```

struct NodoAccion {
    string accion;
    NodoAccion* siguiente;
};

```

```

void push(NodoAccion*& tope, string accion) {
    NodoAccion* nuevo = new NodoAccion{accion, tope};
    tope = nuevo;
}

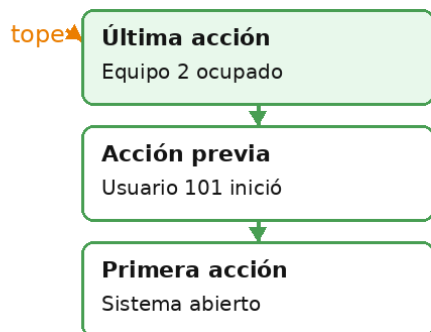
string pop(NodoAccion*& tope) {
    if (tope == nullptr) return "Pila vacia";
    NodoAccion* temp = tope;
    string accion = temp->accion;
    tope = tope->siguiente;
    delete temp;
    return accion;
}

```

Imagen de apoyo visual

c++ Pila dinámica en C++

Pila dinámica con nodos



LIFO

Last In, First Out: la última acción registrada es la primera que se consulta o retira.

```

void push(Nodo*& tope, string accion) {
    Nodo* nuevo = new Nodo{accion, tope};
    tope = nuevo;
}

```

Imagen 6. Pila dinámica: el tope siempre apunta a la acción más reciente.

8. Cola dinámica

Teoría orientada a C++

Una cola dinámica usa nodos enlazados y sigue el principio FIFO: First In, First Out. El primer elemento en entrar es el primero en salir. Para implementarla se usan dos apuntadores: frente y final. Se inserta por el final y se elimina por el frente.

Aplicación PAD-Bienestar: En PAD-Bienestar, esta estructura es ideal para administrar turnos de atención, ya que respeta el orden de llegada y no requiere establecer un límite fijo de personas esperando.

Ejemplo base en C++

```
struct NodoTurno {
    string nombre;
    NodoTurno* siguiente;
};

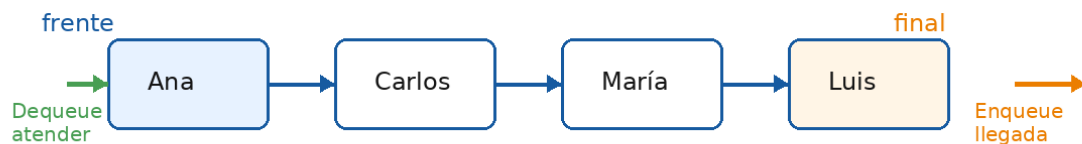
void enqueue(NodoTurno*& frente, NodoTurno*& final, string nombre) {
    NodoTurno* nuevo = new NodoTurno{nombre, nullptr};
    if (final == nullptr) {
        frente = final = nuevo;
    } else {
        final->siguiente = nuevo;
        final = nuevo;
    }
}

string dequeue(NodoTurno*& frente, NodoTurno*& final) {
    if (frente == nullptr) return "Cola vacia";
    NodoTurno* temp = frente;
    string nombre = temp->nombre;
    frente = frente->siguiente;
    if (frente == nullptr) final = nullptr;
    delete temp;
    return nombre;
}
```

Imagen de apoyo visual

c++ Cola dinámica en C++

Cola dinámica con nodos



Aplicación PAD-Bienestar

La cola dinámica organiza turnos sin definir un máximo fijo. Cada nueva persona se agrega al final; la atención siempre ocurre por el frente, respetando el orden de llegada.

Imagen 7. Cola dinámica: frente y final permiten administrar turnos de atención.

9. Modularización y simulación de TDA en C++

Teoría orientada a C++

La modularización consiste en dividir el programa en archivos y funciones. En C++, normalmente se usan archivos .h para declaraciones y archivos .cpp para implementaciones. Este enfoque mejora la lectura, el mantenimiento y la reutilización del código. También permite construir Tipos de Dato Abstracto, donde el programa principal usa operaciones como enqueue o dequeue sin preocuparse por los detalles internos.

Aplicación PAD-Bienestar: En PAD-Bienestar, se puede tener un módulo para la cola de turnos, otro para la lista de sesiones y otro para bitácoras. Así el main.cpp queda más claro y se enfoca en el flujo general del sistema.

Ejemplo base en C++

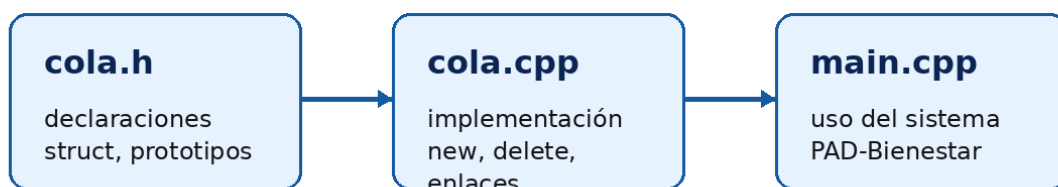
```
// cola.h
struct NodoTurno;
void enqueue(NodoTurno*& frente, NodoTurno*& final, string nombre);
string dequeue(NodoTurno*& frente, NodoTurno*& final);
bool estaVacia(NodoTurno* frente);

// main.cpp
if (!estaVacia(frente)) {
    cout << "Atendiendo a: " << dequeue(frente, final);
}
```

Imagen de apoyo visual

c++ Modularización en C++

Modularización y TDA



Idea clave

Un Tipo de Dato Abstracto oculta los detalles internos y deja al programa principal usar operaciones claras como enqueue(), dequeue(), push() o pop(). En C++ también puede evolucionar a clases.

Imagen 8. Modularización en C++: separar declaraciones, implementación y uso del sistema.

10. Cierre didáctico

La memoria dinámica y las estructuras enlazadas permiten superar la principal limitación de las estructuras estáticas: el tamaño fijo. En C++, el uso de apuntadores, new y delete permite crear nodos durante la ejecución y conectarlos mediante enlaces. Esto hace posible implementar listas enlazadas simples, listas doblemente enlazadas, pilas dinámicas y colas dinámicas.

Para el incidente crítico PAD-Bienestar, estas estructuras son especialmente útiles porque el número de usuarios, sesiones, turnos y eventos puede cambiar constantemente. Una lista puede representar sesiones activas, una cola puede organizar turnos de atención, una pila puede guardar acciones recientes y una lista doble puede permitir navegación en ambos sentidos dentro de un historial.

Conclusión clave: En C++, las estructuras dinámicas ofrecen flexibilidad y mejor aprovechamiento de memoria, pero también exigen responsabilidad: toda memoria creada con new debe liberarse con delete para evitar fugas de memoria y errores con apuntadores colgantes.

Fuentes de apoyo

Universidad Nacional Rosario Castellanos. (2026). Estructura de Datos. Módulo 2: Memoria dinámica y estructuras enlazadas. Contenido Nuclear.

Deitel, P. J., & Deitel, H. M. (2014). C++ Cómo programar (9.^a ed.). Pearson.

Joyanes Aguilar, L. (2017). Fundamentos de programación: algoritmos, estructuras de datos y objetos (4.^a ed.). McGraw-Hill.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2012). Introducción a los algoritmos (3.^a ed.). MIT Press.

Material de apoyo didáctico elaborado para orientar el Módulo 2 al lenguaje C++ y al incidente crítico PAD-Bienestar.